Social Laws in Multi-Agent Planning: A Survey of Compilation-Based Verification

Evgeny Mishlyakov

March 2026

Abstract

A *social law* is a restriction on a multi-agent planning system designed to prevent inter-agent conflicts while preserving each agent's ability to achieve its goal independently. The central property of interest is *robustness*: a social law is robust if every combination of individually valid agent plans can be jointly executed under any adversarial scheduling without deadlock, action failure, or goal violation.

This survey covers three compilation-based methods for verifying robustness. Karpas et al. introduced the reduction to classical planning via an $(n + 1)$-copy state encoding, but the original formulation contains a flaw in deadlock detection. Nir et al. extended and corrected this approach to the numeric setting (MA-SNP), handling numeric fluents and wait-for conditions over numeric constraints. Tuisov et al. generalized further to arbitrary instantaneous-action formalisms with first-order-logic preconditions, using an explicit two-stage execution model.

We present a unified implementation of the numeric and general compilations within the Unified Planning Framework (UPF), together with an automatic translation from restricted-task representations to numeric STRIPS. We evaluate both verifiers on a completed run and report aggregate results on 322 benchmark configurations (644 verifier/configuration pairs). The resulting picture is mixed rather than one-sided: the simple compilation finds easy
counterexamples quickly and is especially strong on `numeric_grid`, while the general compilation proves robustness more often and performs better on domains such as `driverlog`, `zenotravel`, and `numeric_markettrader`. The run also exposes benchmark-specific failure modes, most notably worker failures on `intersection` and internal errors on `numeric_expedition`.

1 Introduction

In a multi-agent planning system, independently acting agents share an environment. Because each agent plans for itself in isolation, their plans may interact destructively: one agent's action may delete a precondition required by another, or two agents may enter a livelock waiting indefinitely for one another.

One principled approach to avoiding such interactions is the *social law* (SL) paradigm [1]. A social law modifies the planning problem available to each agent—removing actions, tightening preconditions, or annotating certain preconditions as *wait-for*—so that conflicts are structurally prevented. A law that succeeds is called *robust*: no matter what individually valid plan each agent chooses, and no matter how an adversarial scheduler interleaves their execution, every agent achieves its goal.

Verifying robustness by direct enumeration is infeasible. Karpas et al. [3] proposed reducing it to classical planning: construct a single-agent planning task Π' whose solutions correspond *exactly* to counterexamples to robustness. Robustness then holds iff Π' is unsolvable—and unsolvability can be certified by a planner.

Two follow-up lines extend this approach. Nir et al. [4] lift the compilation to the *numeric* setting, supporting numeric fluents and wait-for conditions over numeric constraints, and simultaneously fix a bug in the original deadlock-detection logic. Tuisov et al. [5] provide a fully general compilation that is parametric in the action formalism, requiring only that preconditions be computable boolean formulas; this subsumes both classical and numeric STRIPS as special cases.

2 Background

2.1 Multi-Agent Planning Formalisms

MA-STRIPS. A *multi-agent STRIPS task* [2] is a tuple

$$\Pi = \langle F,\, \{A_i\}_{i=1}^n,\, I,\, \{G_i\}_{i=1}^n \rangle,$$

where F is a finite set of Boolean facts, $I \subseteq F$ is the initial state, and for each agent $i \in [n] := \{1, \ldots, n\}$: A_i is agent i's action set and $G_i \subseteq F$ is its goal. Each action $a \in A_i$ is a triple $(pre(a), add(a), del(a))$ with $pre(a), add(a), del(a) \subseteq F$. Applying a in state s (when $pre(a) \subseteq s$) yields $(s \setminus del(a)) \cup add(a)$. The *individual projection* of Π onto agent i is $\Pi_i = \langle F, A_i, I, G_i \rangle$.

Multi-Agent Simple Numeric Planning (MA-SNP). Nir et al. [4] extend MA-STRIPS to the *numeric restricted task* (RT) formalism of Hoffmann [7]. A *multi-agent restricted task* (MART) is

$$\Pi = \langle P,\, V,\, s_0,\, \{A_i\}_{i=1}^n,\, \{G_i\}_{i=1}^n \rangle,$$

where P is a set of propositional variables, V is a set of numeric variables, and $s_0 = \langle s_0^p, s_0^n \rangle$ is the initial state with $s_0^p \subseteq P$ and s_0^n a full numeric assignment. Each action $a \in A_i$ has:

- propositional preconditions $pre_p(a) \subseteq P$,
- numeric preconditions $pre_n(a)$: a set of constraints of the form $v \geq w_0$ with $v \in V$, $w_0 \in \mathbb{Q}$,

- propositional effects $add(a), del(a) \subseteq P$,
- numeric effects $num(a)$: a set of updates $v \mathrel{+}= k_v$ with $k_v \neq 0$.

The full precondition is $pre(a) = pre_p(a) \cup pre_n(a)$.

General Formalism. Tuisov et al. [5] work with a fully general formalism:

$$\Pi = \langle V, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle,$$

where V is a set of variables with arbitrary domains (Boolean, finite-domain, or numeric), and $pre(a)$ is a *computable Boolean formula* over V. The result of applying a to state s is denoted $s[a]$.

2.2 Social Laws and Wait-For Preconditions

Definition 1 (Social Law)**.** A *social law* l is a function that maps a multi-agent task Π to a modified task Π^l. Such modifications may include removing actions, strengthening preconditions, adding auxiliary facts, and annotating preconditions as *wait-for*.

Wait-for semantics. For each action a, its preconditions are partitioned into:

- $pre^f(a)$: *failure preconditions*—if violated during execution, the action fails immediately.
- $pre^w(a)$: *wait-for preconditions*—if violated, the agent blocks until they are satisfied.

The scheduler may only execute action a for agent i when $pre^w(a)$ holds in the current global state.

2.3 Execution Model

Let $\pi^{jnt} = \{\pi_i\}_{i=1}^n$ be a set of individual plans. An execution state is a pair $\langle s, \mathbf{j} \rangle$ where s is the current world state and $\mathbf{j} = (j_1, \ldots, j_n)$ records each agent's position in its plan.

At each step, the scheduler selects an agent i such that $pre^w(\pi_i[j_i])$ holds in s. Four terminal outcomes are possible:

(S) **Success**: all agents finish; $s \models \bigwedge_{i=1}^n G_i$.

(F) **Action Failure**: an agent executes a with $s \not\models pre^f(a)$.

(D) **Deadlock**: no agent can act (pre^w of every pending action is unsatisfied), but some agent is not done.

(G) **Goal Miss**: all agents finish; $s \not\models G_i$ for some i.

Outcomes (F), (D), (G) are collectively *failures*. Let $R^{jnt}(\Pi)$ denote the set of maximal joint executions of Π, and $R^{jnt}_*(\Pi) \subseteq R^{jnt}(\Pi)$ the successful ones.

Definition 2 (Robustness [4])**.** A multi-agent task Π has a *robust* social law iff $R^{jnt}_*(\Pi) \neq \emptyset$ and $R^{jnt}_*(\Pi) = R^{jnt}(\Pi)$.

Equivalently: the social law l applied to Π is robust iff every set of individually valid plans jointly succeeds under every possible scheduling.

3 The Original Compilation (Karpas et al., 2017)

3.1 Construction

Given $\Pi^l = \langle F, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, Karpas et al. construct a classical planning task

$$\Pi' = \langle F', A', I', G' \rangle$$

as follows.

Facts.

$$F' = \underbrace{\{f_i \mid f \in F,\ i \in [n]\}}_{\text{local copies}} \cup \underbrace{\{f_g \mid f \in F\}}_{\text{global copy}} \cup \underbrace{\{wt_{f,i} \mid f \in F,\ i \in [n]\}}_{\text{waiting flags}} \cup \{fin_i \mid i \in [n]\} \cup \{failure,\ act\}.$$

Initial state and goal.

$$I' = \{act\} \cup \{f_i \mid f \in I,\ i \in [n]\} \cup \{f_g \mid f \in I\}.$$
$$G' = \{failure\} \cup \{f^c \mid f \in F\} \cup \{fin_i \mid i \in [n]\}.$$

Action variants. For each $a \in A_i$, three variants are introduced; all apply the effects of a to the local copy i, but differ in their treatment of the global copy.

Success action a_i^s: a succeeds jointly.

$$pre(a_i^s) = act \ \wedge \bigwedge_{f \in pre(a)} (f_i \wedge f_g),$$
$$add(a_i^s) = \{f_i, f_g \mid f \in add(a)\}, \quad del(a_i^s) = \{f_i, f_g \mid f \in del(a)\}.$$

Failure action a_i^f: a non-wait-for precondition of a fails in the global state.

$$pre(a_i^f) = act \ \wedge \bigwedge_{f \in pre(a)} f_i \ \wedge \bigwedge_{f \in pre^w(a)} f_g \ \wedge \bigvee_{f \in pre^f(a)} \neg f_g,$$
$$add(a_i^f) = \{failure\} \cup \{f_i \mid f \in add(a)\}, \quad del(a_i^f) = \{f_i \mid f \in del(a)\}.$$

Wait/deadlock action $a_i^{w,x}$ for each $x \in pre^w(a)$: a's wait-for precondition x fails in the global state.

$$pre(a_i^{w,x}) = act \ \wedge \bigwedge_{f \in pre(a)} f_i \ \wedge \neg x_g,$$
$$add(a_i^{w,x}) = \{failure,\ wt_{x,i}\} \cup \{f_i \mid f \in add(a)\}, \quad del(a_i^{w,x}) = \{f_i \mid f \in del(a)\}.$$

END actions. Two versions per agent: END_i^s (goal met in both copies) and END_i^f (goal met locally but not globally). Both delete act, so no regular actions can fire afterward.

CHECK actions. For each $f \in F$, two post-termination actions verify deadlock genuineness: *CHECK-NO-f* asserts $\neg f_g$, and *CHECK-NO-WAITING-f* asserts $\bigwedge_i \neg wt_{f,i}$. Both add f^c (required for the goal). Together they enforce: if $wt_{f,i}$ is true at the end then f does not hold in the global state.

3.2 Correctness

Theorem 1 (Karpas et al., 2017)**.** Π' is solvable if and only if Π^l is not rationally robust (assuming each Π_i is individually solvable).

The key invariants are: (i) local copy i faithfully tracks agent i's individual execution; (ii) the global copy tracks the adversarially interleaved joint execution; (iii) the CHECK actions certify that waited-for facts remain false forever.

3.3 Deficiency in Deadlock Detection

As noted by Tuisov and Karpas [6], the original formulation contains a subtle error. The wait action $a_i^{w,x}$ fires as soon as x_g is false, *regardless of whether* x could become true before agent i next acts. The CHECK mechanism is supposed to retroactively verify that x truly remains false, but the original encoding does not correctly prevent other agents from satisfying x later under all schedules. This can cause the compilation to accept spurious deadlock witnesses, compromising soundness. The numeric compilation of Nir et al. corrects this by strengthening the success action a_i^s to *forbid* increasing a waited-for numeric variable past its threshold if some other agent is declared waiting for it.

4 Numeric Compilation (Nir et al., 2023)

4.1 Construction

Given a MART $\Pi = \langle P, V, s_0, \{A_i\}_{i=1}^n, \{G_i\}_{i=1}^n \rangle$, the compiled task $\widetilde{\Pi} = \langle \widetilde{P}, \widetilde{V}, \widetilde{A}, \widetilde{s}_0, \widetilde{G} \rangle$ maintains $n+1$ copies of each propositional and numeric variable.

Variables. Let $P_i = \{p_i \mid p \in P\}$ and $V_i = \{v_i \mid v \in V\}$ for $i \in [n] \cup \{g\}$. Waiting flags are:

$$Wt = \{wt_i,\ wt_\psi \mid \psi \in pre^w(a),\ a \in A_i,\ i \in [n]\},$$

where ψ is either a propositional atom or a numeric constraint $v \geq w_0$. Control atoms are:

$$Bk = \{act,\ fail,\ cf\} \cup \{fin_i \mid i \in [n]\}.$$

Thus $\widetilde{P} = Wt \cup Bk \cup \bigcup_{i \in [n] \cup \{g\}} P_i$ and $\widetilde{V} = \bigcup_{i \in [n] \cup \{g\}} V_i$.

Initial state and goal.

$$\widetilde{s}_0^p = \{act\} \cup \{p_i \mid p \in s_0^p,\ i \in [n] \cup \{g\}\}, \quad \widetilde{s}_0^n[v_i] = s_0^n[v] \text{ for all } v \in V,\ i \in [n] \cup \{g\}.$$

$$\widetilde{G} = \{fail\} \cup \{fin_i \mid i \in [n]\}.$$

Four action variants. For each $a \in A_i$, the compilation produces four versions.

(1) Success action a_i^s: applicable when both local and global copies satisfy all preconditions, no agent is in deadlock waiting for a condition that a would violate, and no action conflict has occurred.

$$\begin{aligned}
pre(a_i^s) &= act \wedge \neg wt_i \wedge \neg cf \\
&\quad \wedge \textstyle\bigwedge_{\psi \in pre(a)} (\psi_i \wedge \psi_g) \wedge \bigwedge_{p \in add(a)} \neg wt^p \\
&\quad \wedge \textstyle\bigwedge_{\substack{v \mathrel{+}= k_v \in num(a) \\ w_0 \in W_v}} \big(\neg wt^{v \geq w_0} \vee (v_g < w_0 - k_v)\big), \\
add(a_i^s) &= \{p_i, p_g \mid p \in add(a)\}, \quad del(a_i^s) = \{p_i, p_g \mid p \in del(a)\}, \\
num(a_i^s) &= \{v_i \mathrel{+}= k,\ v_g \mathrel{+}= k \mid v \mathrel{+}= k \in num(a)\}.
\end{aligned}$$

Here $W_v = \{w_0 \mid v \geq w_0 \in \bigcup_{i'} (G_{i'} \cup pre_n(A_{i'}))\}$ is the set of relevant thresholds for variable v. The last precondition line encodes the key **deadlock-preservation invariant**: if some agent declared it is waiting for $v \geq w_0$, then no other agent may increase v past w_0, preventing spurious satisfaction of the waited-for condition.

(2) Fail-action a_i^f: a non-wait-for precondition of a fails in the global copy.

$$\begin{aligned}
pre(a_i^f) &= act \wedge \neg wt_i \wedge \neg cf \wedge \textstyle\bigwedge_{\psi \in pre(a)} \psi_i \wedge \bigwedge_{\psi \in pre^w(a)} \psi_g \wedge \bigvee_{\psi \in pre^f(a)} \neg \psi_g, \\
add(a_i^f) &= \{fail, cf\} \cup \{p_i \mid p \in add(a)\}, \quad del(a_i^f) = \{p_i \mid p \in del(a)\}, \\
num(a_i^f) &= \{v_i \mathrel{+}= k \mid v \mathrel{+}= k \in num(a)\}.
\end{aligned}$$

(3) Deadlock action $a_i^{wt_\varphi}$ for each $\varphi \in pre^w(a)$: the wait-for condition φ is false in the global copy.

$$\begin{aligned}
pre(a_i^{wt_\varphi}) &= act \wedge \neg wt_i \wedge \neg cf \wedge \neg \varphi_g \wedge \textstyle\bigwedge_{\psi \in pre(a)} \psi_i, \\
add(a_i^{wt_\varphi}) &= \{wt_\varphi, fail, wt_i\} \cup \{p_i \mid p \in add(a)\}, \quad del(a_i^{wt_\varphi}) = \{p_i \mid p \in del(a)\}, \\
num(a_i^{wt_\varphi}) &= \{v_i \mathrel{+}= k \mid \ldots\}.
\end{aligned}$$

Setting wt_i permanently marks agent i as waiting; no action can delete wt_i. The deadlock-preservation invariant in a_i^s ensures φ_g can never later become true once wt_φ is set.

(4) Local action a_i^l: applicable after a deadlock or action conflict; updates only the local copy.

$$\begin{aligned}
pre(a_i^l) &= act \wedge (wt_i \vee cf) \wedge \textstyle\bigwedge_{\psi \in pre(a)} \psi_i, \\
add(a_i^l) &= \{p_i \mid p \in add(a)\}, \quad del(a_i^l) = \{p_i \mid p \in del(a)\}, \quad num(a_i^l) = \{v_i \mathrel{+}= k \mid v \mathrel{+}= k \in num(a)\}.
\end{aligned}$$

Local actions allow each agent to complete a valid individual plan in its local copy after failure, supplying the required fin_i atoms.

END actions. END_i^s: agent i achieved G_i in both copies (adds fin_i, deletes act). END_i^f: agent i achieved G_i locally but not globally (additionally adds $fail$).

4.2 Correctness

Theorem 2 (Nir et al., 2023)**.** $\widetilde{\Pi}$ is solvable $\iff$ Π is not robust.

The proof has three lemmas. (L1) Any plan π' for $\widetilde{\Pi}$ projects to a valid individual plan π_i for each agent i. (L2) The sequence of global-copy actions in π' constitutes a failed joint execution. (L3) Conversely, any failed joint execution can be lifted to a plan for $\widetilde{\Pi}$ by substituting the appropriate action variants and appending local-copy completions.

The numeric setting introduces a key undecidability caveat: numeric planning is undecidable in general [8], so the unsolvability witness required for positive robustness results cannot always be found. In practice, counterexamples are found quickly when the law is not robust, and several domains admit manual proofs of unsolvability.

5 General Compilation (Tuisov et al., 2025)

5.1 Design Principles

The general compilation [5] removes the dependence on a specific representation format. Its two key changes relative to the numeric compilation are:

1. **Preconditions as arbitrary boolean formulas.** The compiled task uses the same variable types as the input; no translation to STRIPS or numeric STRIPS is required.

2. **Explicit two-stage execution.** Stage 1 simulates joint execution; stage 2 handles bookkeeping (completing individual plans in local copies). This replaces the act/cf flag mechanism with a cleaner $stage1/stage2$ split.

5.2 Construction

Given $\Pi = \langle V, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, define the local variable copies $V_i = \{v_i \mid v \in V\}$ and global copy $V_g = \{v_g \mid v \in V\}$. The auxiliary Boolean flags are:

$$Aux = \{conflict,\ stage1,\ stage2,\ PrV,\ PoD\} \cup \{allow(a) \mid a \in \textstyle\bigcup_i A_i\} \cup \{RA_i,\ fin_i \mid i \in [n]\}.$$

The compiled variables are $V' = Aux \cup V_g \cup \bigcup_{i \in [n]} V_i$.

For a set of conditions X over V, write X_i and X_g for the same conditions over V_i and V_g respectively.

Five action variants per action. For each $a_i \in A_i$, the compilation defines $A'_i = \{a_i^s, a_i^f, a_i^w, a_i^l, \textit{deadlock-}a_i\}$.

(1) Success a_i^s: applicable in stage 1 when all preconditions hold in both copies; advances the global state.

$$\begin{aligned} \textit{pre}(a_i^s) &= \textit{stage1} \wedge (\textit{allow}(a_i) \vee \neg RA_i) \wedge \textit{pre}_i(a_i) \wedge \textit{pre}_g(a_i), \\ \textit{eff}(a_i^s) &= \textit{eff}_i(a_i) \cup \textit{eff}_g(a_i) \cup \textstyle\bigcup_{a \in A_i} \{\textit{allow}(a)\}. \end{aligned}$$

(2) Failure a_i^f: applicable in stage 1 when wait-for conditions hold globally but failure preconditions do not; terminates stage 1 and records a precondition violation (PrV).

$$\begin{aligned} \textit{pre}(a_i^f) &= \textit{stage1} \wedge (\textit{allow}(a_i) \vee \neg RA_i) \wedge \textit{pre}_i(a_i) \wedge \textit{pre}_g^w(a_i) \wedge \neg \textit{pre}_g^f(a_i), \\ \textit{eff}(a_i^f) &= \{PrV,\ \textit{stage2},\ \neg \textit{stage1}\} \cup \textit{eff}_i(a_i). \end{aligned}$$

(3) Wait a_i^w: applicable in stage 1 when wait-for conditions fail in the local copy; locks agent i to only action a_i by disabling all other actions in A_i.

$$\begin{aligned} \textit{pre}(a_i^w) &= \textit{stage1} \wedge \textit{allow}(a_i) \wedge \textit{pre}_i(a_i) \wedge \neg \textit{pre}_i^w(a_i), \\ \textit{eff}(a_i^w) &= \textstyle\bigcup_{a \neq a_i} \{\neg \textit{allow}(a)\} \cup \{RA_i\}. \end{aligned}$$

(4) Deadlock declaration *deadlock-*a_i: certifies that agent i is permanently stuck; ends stage 1 and sets a possible-deadlock flag (PoD).

$$\begin{aligned} \textit{pre}(\textit{deadlock-}a_i) &= \textit{allow}(a_i) \wedge \neg \textit{pre}_g^w(a_i) \wedge RA_i, \\ \textit{eff}(\textit{deadlock-}a_i) &= \{\textit{fin}_i,\ PoD,\ \neg \textit{stage1}\}. \end{aligned}$$

(5) Local a_i^l: applicable in stage 2; updates only local copy i so agent i can complete its individual plan.

$$\begin{aligned} \textit{pre}(a_i^l) &= \textit{stage2} \wedge (\textit{allow}(a_i) \vee \neg RA_i) \wedge \textit{pre}_i(a_i), \\ \textit{eff}(a_i^l) &= \textit{eff}_i(a_i) \cup \{\textit{allow}(a_i),\ \neg RA_i\}. \end{aligned}$$

Auxiliary actions.

- *end-success*$_i$: applicable when G_i^g holds; sets $\textit{fin}_i$ and deletes *stage1*.
- *start-stage2*: applicable when $\bigwedge_i \textit{fin}_i$; sets *stage2*, clears *stage1*.
- *declare-deadlock*: applicable in stage 2 with PoD and all local goals met; sets *conflict*.
- *declare-fail*: applicable in stage 2 with PrV and all local goals met; sets *conflict*.
- *goals-not-achieved*: applicable in stage 2 when all local goals hold but $\neg G^g$; sets *conflict*.

Initial state and goal.

$$I' = \{stage1\} \cup \{I[v_i] = I[v] \mid v \in V,\, i \in [n]\} \cup \{I[v_g] = I[v] \mid v \in V\},$$

all other flags false. The goal is $G' = \{conflict\}$.

5.3 Correctness

Theorem 3 (Tuisov et al., 2025)**.** Π' is solvable $\iff$ Π is not robust.

The proof establishes an explicit bijection between failed executions $\pi^{[n]}$ in Π and plans π' for Π' via an equivalence relation on state sequences. The three failure cases (goal miss, action failure, deadlock) are handled separately, with the stage 1 prefix $\mathrm{eq}(\pi^{[n]})$ constructed by substituting action superscripts appropriately, and the stage 2 suffix consisting of local-copy completions and one of the three failure-declaration actions.

5.4 Relationship to Prior Compilations

Table 1 summarizes the structural differences between the three compilations.

Table 1: Comparison of social law verification compilations.

	Karpas et al.	**Nir et al.**	**Tuisov et al.**
Input formalism	MA-STRIPS	MA-SNP (MART)	General (instantaneous)
Output formalism	Classical STRIPS	Numeric STRIPS	Same as input
Action variants	3 (a^s, a^f, a^w)	4 ($+a^l$)	5 ($+$*deadlock-a*)
Deadlock handling	CHECK actions	*wt* flags + a^s guard	a^w + *deadlock-a*
Stage separation	Implicit (*act* flag)	Implicit (*act* flag)	Explicit (stage1/stage2)
Bug fix	—	Yes (deadlock preservation)	Subsumed

6 Implementation and Empirical Evaluation

6.1 Implementation

Both the numeric compilation of Nir et al. and the general compilation of Tuisov et al. are implemented in Python on top of the Unified Planning Framework (UPF) [12]. A preprocessing step automatically translates restricted tasks (MA-SNP/MART) to numeric STRIPS when applicable, enabling the numeric compilation to be applied when the problem fits that fragment.

The empirical section below is based on the completed evaluation run. The full run contains 342 benchmark configurations and executes both verifiers on each of them, yielding 684 verifier/configuration pairs. The filtered analysis reported here covers 302 configurations and 604 verifier/configuration pairs. All runs used the ENHSP backend through UPF with a planner timeout of 900 seconds, a wall-clock limit of 1800 seconds, a CPU limit of 1800 seconds, and a memory cap of 16 GB.

Table 2: Completed instances in the completed run, split by verifier and social-law setting.

Domain	without_sl general	without_sl simple	with_sl general	with_sl simple
driverlog	15	10	15	10
grid	20	20	20	20
intersection	8	8	8	8
numeric_expedition	4	6	2	2
numeric_grid	4	20	8	20
numeric_markettrader	9	0	0	0
numeric_zenotravel	5	3	5	0
zenotravel	12	6	10	7
Total	77	73	68	67

Table 3: Aggregate completion summary.

Verifier	without_sl completed	with_sl completed	Total completed	Completed within 30s
general	77	68	145	116
simple	73	67	140	126

6.2 Benchmark Domains

The benchmark suite spans nine domains: `driverlog`, `grid`, `intersection`, `numeric_expedition`, `numeric_grid`, `numeric_markettrader`, `numeric_zenotravel`, and `zenotravel`. Most domains were evaluated both with and without a social law; `numeric_markettrader` appears only without a social law in this run. Most domain/setting combinations contain 20 instances; `numeric_expedition` contains 21 because the suite also includes a minimal instance.

6.3 Results

Table 2 reports, for each domain and social-law setting, how many instances each verifier completed before timeout or execution failure. Here "completed" means that the run ended with either a concrete counterexample (`SOLVED_SATISFICING`) or a proof of robustness (`UNSOLVABLE_PROVEN`).

Figure 1 shows the cumulative number of completed pairs under increasing time thresholds. The simple verifier retains a clear early advantage: within 30 seconds it completes 126 pairs, compared to 116 for the general verifier. As the time budget grows, however, the general verifier catches up and ends slightly ahead overall, finishing with 145 completed pairs versus 140 for the simple verifier.

To make the split by verifier and social-law setting visible, Figure 2 reproduces the four solved-under-time plots generated directly from the run logs.

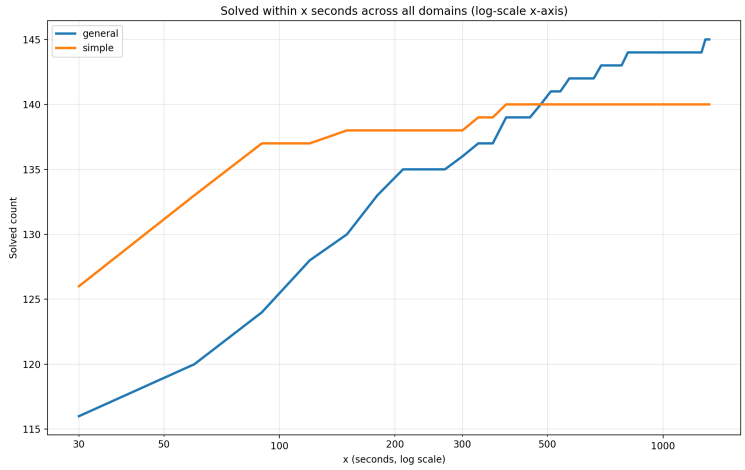

Figure 1: Cumulative completed verifier/configuration pairs under increasing time thresholds.

6.4 Discussion

The run does not support a single global winner; instead it exposes a domain-dependent trade-off between the two implementations.

Across the filtered set of 322 configurations, the general verifier completes 146 cases and the simple verifier completes 141. This does not erase the speed advantage of the simple verifier on easy cases: it still reaches 127 completed pairs within 30 seconds, compared to 117 for the general verifier. But beyond the earliest portion of the curve, the general verifier catches up and ultimately finishes ahead.

Domain behavior is strongly heterogeneous. Several domains clearly favor the general verifier: it completes 15 vs. 10 instances on both `driverlog` settings, 12 vs. 6 and 10 vs. 7 on `zenotravel`, and it is the only verifier to solve any `numeric_markettrader` instances (9 without a social law). In contrast, the simple verifier dominates `numeric_grid`, solving all 20 instances both with and without a social law, whereas the general verifier solves only 8 and 4 respectively. Some domains are essentially neutral in this run, notably `grid` and `intersection`, where both verifiers have identical completion counts.

7 Conclusion

We have surveyed three compilation-based approaches to verifying social law robustness in multi-agent planning. The central technical insight shared by all three is the $(n+1)$-copy encoding: maintain one local copy per agent (tracking

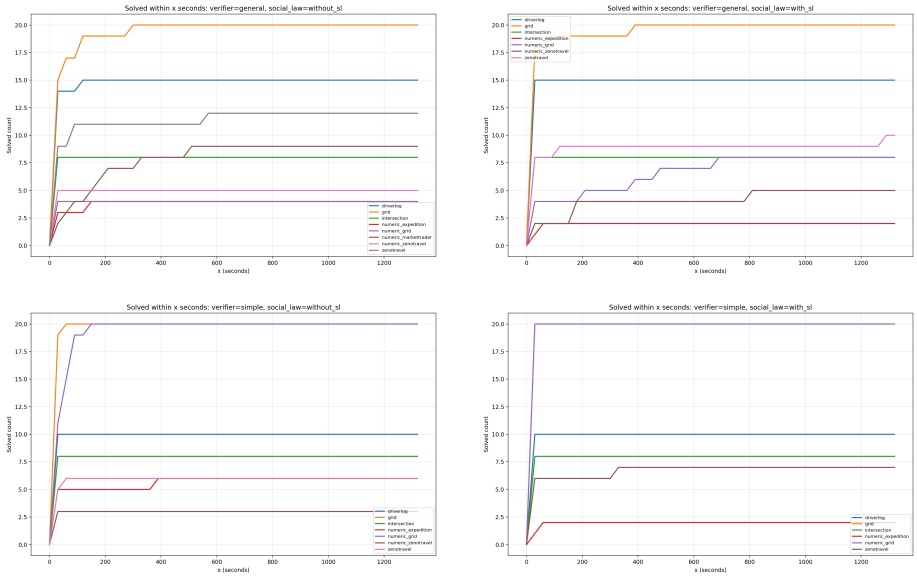

Figure 2: Solved-under-time curves split by verifier and by whether a social law is enabled. Top row: general verifier. Bottom row: simple verifier. Left: without social law. Right: with social law.

individual execution) and one global copy (tracking the adversarial joint execution), then search for a configuration that succeeds locally for every agent but fails globally.

The main technical advances across the sequence of works are: (i) the extension from Boolean to numeric state (Nir et al.), which also fixes a deadlock-preservation error in the original; (ii) the generalization to arbitrary instantaneous-action formalisms via explicit stage separation (Tuisov et al.), which subsumes the earlier compilations and opens the door to social laws in richer planning models.

The empirical picture from our completed run is more nuanced than the simplified story often told in the prior papers. Neither verifier dominates overall. The simple verifier has a real speed advantage on easy counterexample-rich cases and performs extremely well on `numeric_grid`; the general verifier is much stronger at proving robustness and performs better on several harder domains, including `driverlog`, `zenotravel`, and `numeric_markettrader`. At the same time, the run reveals concrete engineering issues—especially `intersection` worker failures and `numeric_expedition` internal errors—that should be addressed before treating the present coverage numbers as a definitive comparison.

References

[1] M. Tennenholtz and Y. Moses. On cooperation in a multi-entity model. In *IJCAI*, pages 918–923, 1989.

[2] R. I. Brafman and C. Domshlak. From one to many: Planning for loosely coupled multi-agent systems. In *ICAPS*, pages 28–35, 2008.

[3] E. Karpas, A. Shleyfman, and M. Tennenholtz. Automated verification of social law robustness in STRIPS. In *ICAPS*, pages 163–171, 2017.

[4] R. Nir, A. Shleyfman, and E. Karpas. Automated verification of social laws in numeric settings. In *AAAI*, pages 12087–12094, 2023.

[5] A. Tuisov, E. Mishlyakov, A. Shleyfman, and E. Karpas. Towards a unified view of social laws with instantaneous actions. In *IJCAI*, 2025.

[6] A. Tuisov and E. Karpas. Automated verification of social law robustness for reactive agents. In *ECAI*, volume 325, pages 2386–2393, 2020.

[7] J. Hoffmann. The metric-FF planning system: Translating "ignoring delete lists" to numeric state variables. *JAIR*, 20:291–341, 2003.

[8] M. Helmert. Decidability and undecidability results for planning with numerical state variables. In *AIPS*, pages 303–312, 2002.

[9] M. Helmert. New complexity results for classical planning benchmarks. In *ICAPS*, pages 52–61, 2006.

[10] E. Scala et al. Subgoaling techniques for satisficing and optimal numeric planning. *JAIR*, 68:691–752, 2020.

[11] M. Stolba, A. Komenda, and D. L. Kovacs. Competition of distributed and multiagent planners (CoDMAP). *http://agents.fel.cvut.cz/codmap/*, 2015.

[12] Unified Planning Framework. Unified planning: Modeling, manipulating and solving AI planning problems in Python. *SoftwareX*, 29:102012, 2025.